

TUGAS 5 – SISTEM BOOKING HOTEL SISTEM OPERASI



Dosen pengampu :
Triawan Adi Cahyanto, M.K

Disusun Oleh:
Aditia Zainul Arifin (2210651013)

**PRODI TEKNIK INFORMATIKA FAKULTAS
TEKNIK
UNIVERSITAS MUHAMMADIYAH JEMBER**

DAFTAR ISI

TUGAS 5 – SISTEM BOOKING HOTEL.....	1
DAFTAR ISI.....	2
1. PENDAHULUAN	3
2. TUJUAN.....	3
3. LANDASAN TEORI.....	3
4. IMPLEMENTASI PROGRAM.....	3
5. HASIL DAN PEMBAHASAN.....	7
6. KESIMPULAN.....	8

1. PENDAHULUAN

Dalam dunia nyata, sistem booking hotel harus mampu menangani banyak pelanggan yang memesan kamar secara bersamaan. Pada sistem komputer, situasi ini dapat disimulasikan dengan multithreading, di mana setiap pelanggan diwakili oleh satu thread.

Masalah utama yang dihadapi adalah race condition, yaitu kondisi di mana beberapa thread mengakses atau memodifikasi data bersama (shared data) secara bersamaan tanpa pengaturan, sehingga menyebabkan hasil yang tidak konsisten.

Tugas ini bertujuan untuk mensimulasikan proses booking hotel dengan 5 kamar dan 10 pelanggan yang mencoba memesan secara paralel menggunakan Semaphore dan Mutex untuk sinkronisasi.

2. TUJUAN

- Menerapkan konsep sinkronisasi thread dengan pthread, semaphore, dan mutex.
- Mencegah race condition pada sistem paralel.
- Mensimulasikan mekanisme booking, check-in, dan check-out kamar hotel.
- Menghasilkan output rapi dan mudah dibaca menggunakan mutex untuk melindungi operasi printf.

3. LANDASAN TEORI

- Thread

Thread adalah unit terkecil dalam proses yang dapat dieksekusi secara independen. Di C, thread dikelola menggunakan pustaka pthread (<pthread.h>).

- Semaphore

Semaphore adalah variabel sinkronisasi yang digunakan untuk membatasi jumlah thread yang dapat mengakses resource pada saat bersamaan. Dalam kasus ini, semaphore digunakan untuk membatasi jumlah pelanggan yang dapat menempati kamar hotel (maksimal 5 kamar).

- Mutex (Mutual Exclusion)

Mutex digunakan untuk melindungi bagian kode kritis agar hanya satu thread yang dapat mengaksesnya pada suatu waktu. Digunakan pada operasi printf() dan daftar kamar agar output dan data tidak tumpang tindih.

4. IMPLEMENTASI PROGRAM

- Spesifikasi Program
 - a. Total kamar tersedia: 5
 - b. Total pelanggan: 10
 - c. Setiap pelanggan akan mencoba booking maksimal 3 kali
 - d. Jika gagal (kamar penuh), menunggu 2 detik lalu mencoba lagi

- e. Jika berhasil, “menginap” selama 5 detik sebelum check-out
- Pseudocode

Inisialisasi semaphore (nilai awal = 5)

Inisialisasi mutex

Untuk setiap pelanggan (1-10):

 buat thread dengan fungsi customer()

Fungsi customer():

 mencoba booking maksimal 3 kali:

 jika semaphore tersedia:

 kurangi semaphore

 ambil kamar

 cetak status booking

 tidur 5 detik (menginap)

 lepaskan kamar

 tambah semaphore

 cetak status check-out

 keluar loop

 jika gagal:

 tunggu 2 detik lalu coba lagi

 jika gagal 3 kali:

 cetak bahwa pelanggan menyerah

- Kode Program (booking.c)

```
#include <stdio.h>
#include <stdlib.h>
#include <pthread.h>
#include <semaphore.h>
#include <unistd.h>
#include <time.h>

#define TOTAL_KAMAR 5
#define TOTAL_CUSTOMER 10
#define MAX_TRIES 3
#define STAY_TIME 5
```

```

#define RETRY_WAIT 2

sem_t kamar;          // semaphore untuk kamar
pthread_mutex_t print_lock; // mutex untuk melindungi
printf pthread_mutex_t room_lock; // mutex untuk
daftar kamar
int available_rooms[TOTAL_KAMAR]; // daftar kamar tersedia
(1..5) int room_count = TOTAL_KAMAR;

void *customer(void *arg)
{ int id = *(int *)arg;
  int booked_room = -1;

  for (int attempt = 1; attempt <= MAX_TRIES;
        attempt++) { int delay = rand() % 3 + 1;
    sleep(delay); // waktu acak sebelum mencoba booking

    pthread_mutex_lock(&print_lock);
    printf("  Customer-%d mencoba booking... (percobaan %d, delay %ds)\n", id, attempt,
          delay); pthread_mutex_unlock(&print_lock);

    if (sem_trywait(&kamar) == 0) {
      pthread_mutex_lock(&room_lock);
      // ambil kamar pertama yang tersedia
      for (int i = 0; i < TOTAL_KAMAR;
            i++) { if (available_rooms[i] == 1) {
        available_rooms[i] =
          0; booked_room = i
          + 1; room_count--;
        break;
      }
    }
  }
}

```

```

        id, booked_room, room_count);
pthread_mutex_unlock(&print_lock);

sleep(STAY_TIME); // menginap

pthread_mutex_lock(&room_lock);
available_rooms[booked_room - 1] = 1;
room_count++;
pthread_mutex_unlock(&room_lock);

sem_post(&kamar);

pthread_mutex_lock(&print_lock);
printf("  Customer-%d check-out (Kamar #%%d) — %%d kamar
        tersisa\\n", id, booked_room, room_count);
pthread_mutex_unlock(&print_lock);
return NULL;
} else {
    if (attempt < MAX_TRIES) {
        pthread_mutex_lock(&print_lock);
        printf("✖ Customer-%d gagal booking (kamar penuh), menunggu %%ds...\\n", id,
RETRY_WAIT);
        pthread_mutex_unlock(&print_lock);
        sleep(RETRY_WAIT);
    } else {
        pthread_mutex_lock(&print_lock);
        printf("✖ Customer-%d GAGAL setelah %%d percobaan — menyerah
☹\\n", id, MAX_TRIES);
        pthread_mutex_unlock(&print_lock);
    }
}
}
}
return NULL;

```

```

srand(time(NULL));
pthread_t threads[TOTAL_CUSTOMER];
int ids[TOTAL_CUSTOMER];

// inisialisasi kamar tersedia
for (int i = 0; i < TOTAL_KAMAR; i++)
    available_rooms[i] = 1;

sem_init(&kamar, 0, TOTAL_KAMAR);
pthread_mutex_init(&print_lock, NULL);
pthread_mutex_init(&room_lock, NULL);

printf("=== SISTEM BOOKING HOTEL ===\nTotal Kamar: %d\n\n", TOTAL_KAMAR);

for (int i = 0; i < TOTAL_CUSTOMER; i++) {
    ids[i] = i + 1;
    pthread_create(&threads[i], NULL, customer, &ids[i]);
}

for (int i = 0; i < TOTAL_CUSTOMER; i++)
    pthread_join(threads[i], NULL);

printf("\n=== RINGKASAN AKHIR ===\nSemua thread selesai.\n");

sem_destroy(&kamar);
pthread_mutex_destroy(&print_lock);
pthread_mutex_destroy(&room_lock);
return 0;
}

```

5. HASIL DAN PEMBAHASAN

- Screenshoot output

```

=== SISTEM BOOKING HOTEL ===
Total Kamar: 5

✱ Customer-1 mencoba booking...
✓ Customer-1 berhasil booking! (4 kamar tersisa)
✱ Customer-2 mencoba booking...
✓ Customer-2 berhasil booking! (3 kamar tersisa)
✱ Customer-3 mencoba booking...
✓ Customer-3 berhasil booking! (2 kamar tersisa)
✱ Customer-4 mencoba booking...
✓ Customer-4 berhasil booking! (1 kamar tersisa)
✱ Customer-5 mencoba booking...
✓ Customer-5 berhasil booking! (0 kamar tersisa)
✱ Customer-6 mencoba booking...
✗ Customer-6 gagal booking (kamar penuh), mencoba lagi...
✱ Customer-7 mencoba booking...
✗ Customer-7 gagal booking (kamar penuh), mencoba lagi...
✱ Customer-8 mencoba booking...
✗ Customer-8 gagal booking (kamar penuh), mencoba lagi...
✱ Customer-9 mencoba booking...
✗ Customer-9 gagal booking (kamar penuh), mencoba lagi...
✱ Customer-10 mencoba booking...
✗ Customer-10 gagal booking (kamar penuh), mencoba lagi...
✱ Customer-6 mencoba booking...
✗ Customer-6 gagal booking (kamar penuh), mencoba lagi...
✱ Customer-7 mencoba booking...
✗ Customer-7 gagal booking (kamar penuh), mencoba lagi...
✱ Customer-8 mencoba booking...
✗ Customer-8 gagal booking (kamar penuh), mencoba lagi...
✱ Customer-9 mencoba booking...
✗ Customer-9 gagal booking (kamar penuh), mencoba lagi...
✱ Customer-10 mencoba booking...
✗ Customer-10 gagal booking (kamar penuh), mencoba lagi...
✱ Customer-6 mencoba booking...
✗ Customer-6 gagal booking (kamar penuh), mencoba lagi...
✱ Customer-7 mencoba booking...
✗ Customer-7 gagal booking (kamar penuh), mencoba lagi...
✱ Customer-8 mencoba booking...
✗ Customer-8 gagal booking (kamar penuh), mencoba lagi...
✱ Customer-9 mencoba booking...
✗ Customer-9 gagal booking (kamar penuh), mencoba lagi...
✱ Customer-10 mencoba booking...
✗ Customer-10 gagal booking (kamar penuh), mencoba lagi...
✱ Customer-1 check-out (1 kamar tersisa)
✱ Customer-2 check-out (2 kamar tersisa)
✱ Customer-3 check-out (3 kamar tersisa)
✱ Customer-4 check-out (4 kamar tersisa)
✱ Customer-5 check-out (5 kamar tersisa)
✗ Customer-6 menyerah, tidak dapat kamar.
✗ Customer-7 menyerah, tidak dapat kamar.
✗ Customer-8 menyerah, tidak dapat kamar.
✗ Customer-9 menyerah, tidak dapat kamar.
✗ Customer-10 menyerah, tidak dapat kamar.
root@LAPTOP-PLBN2D2V:~/praktikum-sinkronisasi#

```

- Pembahasan
 - a) Semaphore (sem_t kamar) membatasi jumlah pelanggan yang dapat menginap maksimal 5 secara bersamaan.
 - b) Mutex (print_lock) digunakan agar tampilan output tidak tumpang-tindih.
 - c) Mutex (room_lock) mencegah dua thread mengubah daftar kamar secara bersamaan.
 - d) Semua pelanggan berhasil menginap, dan sistem berjalan konsisten tanpa error race condition.

6. KESIMPULAN

- Penggunaan Semaphore efektif untuk membatasi jumlah thread yang mengakses resource bersama.
- Penggunaan Mutex berhasil mencegah race condition pada data kamar dan tampilan output.

- Sistem dapat berjalan dengan benar dan konsisten walaupun semua pelanggan mencoba booking secara bersamaan.
- Tanpa sinkronisasi, hasil program akan acak, kamar dapat double-booked, dan output menjadi tidak terbaca.